



Artificial Intelligence Foundations (CIS4049-N)

Lab Introduction

Module Leader: Dr Alessandro Di Stefano

Email: A.DiStefano@tees.ac.uk

School of **Computing, Engineering & Digital Technologies**

tees.ac.uk/computing

Lab Session - Introduction

- **Reinforcement Learning in R**
 - *ReinforcementLearning* package
- **Reinforcement Learning with Python**

RL with R – *ReinforcementLearning* package



- *ReinforcementLearning* package, which allows one to perform model-free reinforcement in R.
- The implementation uses input data in the form of sample sequences consisting of **states**, **actions**, and **rewards**.
- Based on such training examples, the package allows a reinforcement learning agent to learn an **optimal policy** that **defines the best possible action in each state**.
- **Step-by-step examples** on how using the ReinforcementLearning package.
- **Methods** to **customize** the learning and action selection behaviour of the agent.

RL with R – *ReinforcementLearning* package



- Learning an **optimal policy** from a fixed set of a priori known transition samples.
- **Predefined learning rules** and **action selection modes**.
- A highly customizable framework for **model-free reinforcement learning tasks**.

RL with R – *ReinforcementLearning* package



- **Data preparation** - The training examples for reinforcement learning can:
 - (1) be collected from an external source and inserted into a tabular data structure (learning from pre-defined observations)
 - (2) **generated dynamically** by querying a function that defines the behaviour of the environment (Dynamic learning from an interactive environment function)
- **Learning phase:**
 - **General setup:** routine ***ReinforcementLearning()*** bundles the main functionality, which teaches a reinforcement learning agent using the previous input data.
 - *Data* argument that must be a data frame object in which each row represents a state transition tuple and the user is required to **specify the column names** of the individual tuple elements within *data*.
 - **Learning parameters configuration** – **alpha** (learning rate), **gamma** (discount factor), **epsilon** (exploration parameter), **iter** (# of repeated learning iterations)

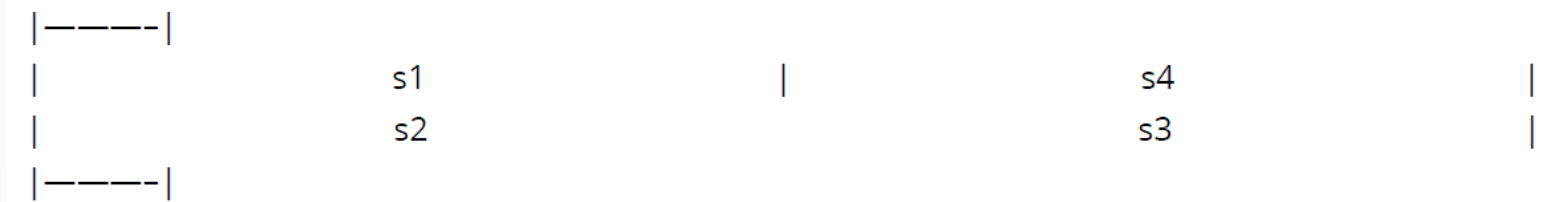
RL with R – *ReinforcementLearning* package



- **Diagnostics** - The result of the learning process is an object of type *rl* that contains the **state-action table** and an **optimal policy** with the **best possible action in each state**.
 - The command *computePolicy(model)* shows the **optimal policy**
 - *print(model)* outputs the state-action table, i.e. the Q-value of each state-action pair.

Working Example – *GridWorld*

- Example – **Gridworld** - robot in a **grid-shaped maze** (adapted from Sutton (1998)).
- The agent must navigate from a random starting position to a final position on a simulated **2×2 grid**.



- **Each cell** on the grid reflects **one state**, yielding a total of **4 different states**.
- In each state, the agent can perform **one out of four possible actions**, i.e., **to move up, down, left, or right**, with the only restriction being that it must remain on the grid.
- In other words, the grid is surrounded by a **wall**, which makes it impossible for the agent to move off the grid.
- A wall between *s1* and *s4* hinders direct movements between these states.
- **Rewards** - each movement leads to a negative reward of **-1** to penalize routes that are not the shortest path. If the agent reaches the goal position, it earns a reward of **10**.

Working Example – *GridWorld*

- Defining the environment function
- Learning an optimal policy
- Evaluating policy learning
 - ***computePolicy(model)*** to display the policy that defines the best possible action in each state.
 - ***print(model)*** in order to write the entire state-action table to the screen, i.e., the Q-value of each state-action pair.
 - ***summary(model)*** to inspect the model further. This command outputs additional diagnostics regarding the model such as the number of states and actions. Analyse the distribution of rewards.
- **Applying a policy to unseen data** - apply an existing policy to unseen data to evaluate the out-of-sample performance of the agent.
- **Updating an existing policy with new observational data** - beneficial when additional data points become available or when one wants to plot the reward as a function of the number of training samples

Working Example – *TicTacToe*

- Underlining the capabilities of the *ReinforcementLearning* package when using **state-transition tuples** from an external source without the need for modelling the dynamics of the environment.
- The following example utilizes the dataset containing game states of randomly sampled **tic-tac-toe games** to **learn the optimal actions for each state of the board** (adapted from Sutton (1998)).
- All states are observed from the perspective of **player X** who is also **assumed to have played first**. The player who succeeds in placing three of their marks in a horizontal, vertical, or diagonal row wins the game. **Reward** for **player X** is **+1** for ‘win’, **0** for ‘draw’, and **-1** for ‘loss’.
- The **current state of the board** is represented by a row-wise concatenation of the players’ marks in a **3x3 grid**. For example, “.....X.B” denotes a board state in which player X has placed a mark in the first field of the third column whereas player B has placed a mark in the third field of the third column.

```
## .....X.B
## | . | . | . |
## |-----|
## | . | . | . |
## |-----|
## | X | . | B |
```

Notes – RL package

- **Q-learning** is guaranteed to **converge** to an **optimal policy**, but the **method** is **computationally demanding** as it relies on **continuous interactions between an agent and its environment**
- **Batch reinforcement learning**
 - mitigates the ‘exploration overhead’ problem in pure online learning (see http://machine-learning-lab.com/_media/publications/langegabelriedmiller2011chapter.pdf)
 - In combination with **experience replay**, it **speeds up convergence** by collecting and replaying observed state transitions repeatedly to the agent as if they were new observations collected while interacting with the system.
 - Nonetheless, due to the fact that the package is written purely in R, the **applicability** of the package to very large-scale problems (such as applications from computer vision) is still **limited**

Pros— RL package

- 1. Learning optimal strategies** for real-world problems with limited state and action sets (e.g. finding optimal strategies for simple games, training a simple stock market trading agent, learning polarity labels in applications from NLP).
- 2. Speed up performance** by adjusting learning parameters and making use of **experience replay**.
- 3. Train an agent from pre-defined observations without the need for modelling the dynamics of the environment.** Speeding up convergence and useful when the state-transition tuples have been collected from an external source (i.e., sensor data).
- 4. Provides a highly customizable framework for model-free reinforcement learning tasks** in which the **functionality** can **easily be extended**. For example, users may attempt to speed up performance by defining alternative reinforcement learning algorithms and integrating them into the package code.

Cons – RL package

1. **Solving large-scale problems with high-dimensional state-action spaces**, such as those from computer vision (users may consider reinforcement learning implementations written in “faster” programming languages).
2. **Solving reinforcement learning problems requiring real-time interaction** (e.g., real-time interaction with a robot).

Reinforcement Learning in Python

- **Q-Learning in Python with Open AI Gym**

- <https://www.learndatasci.com/tutorials/reinforcement-q-learning-scratch-python-openai-gym/>
- <https://pythonprogramming.net/q-learning-reinforcement-learning-python-tutorial/>
- <https://gym.openai.com/docs/>

- **Grid-world environment**

- <https://medium.com/analytics-vidhya/a-simple-reinforcement-learning-environment-from-scratch-72c37bb44843>

Summing-up

- **Reinforcement Learning with R** - *ReinforcementLearning* package
- **Working Examples** - *GridWorld* and *TicTacToe*
- **Reinforcement Learning with Python**

Questions?

A.DiStefano@tees.ac.uk